Build, Break, Defend: Securing the Model Context Protocol (MCP) in Action

Join the workshop
Slack channel

- Shruti Datta Gupta
- Spandana Gorantla

Hello!

Shruti Datta Gupta
Product Security Engineer
Adobe

Spandana Gorantla
Product Security Engineer
Adobe

Just Ship It!

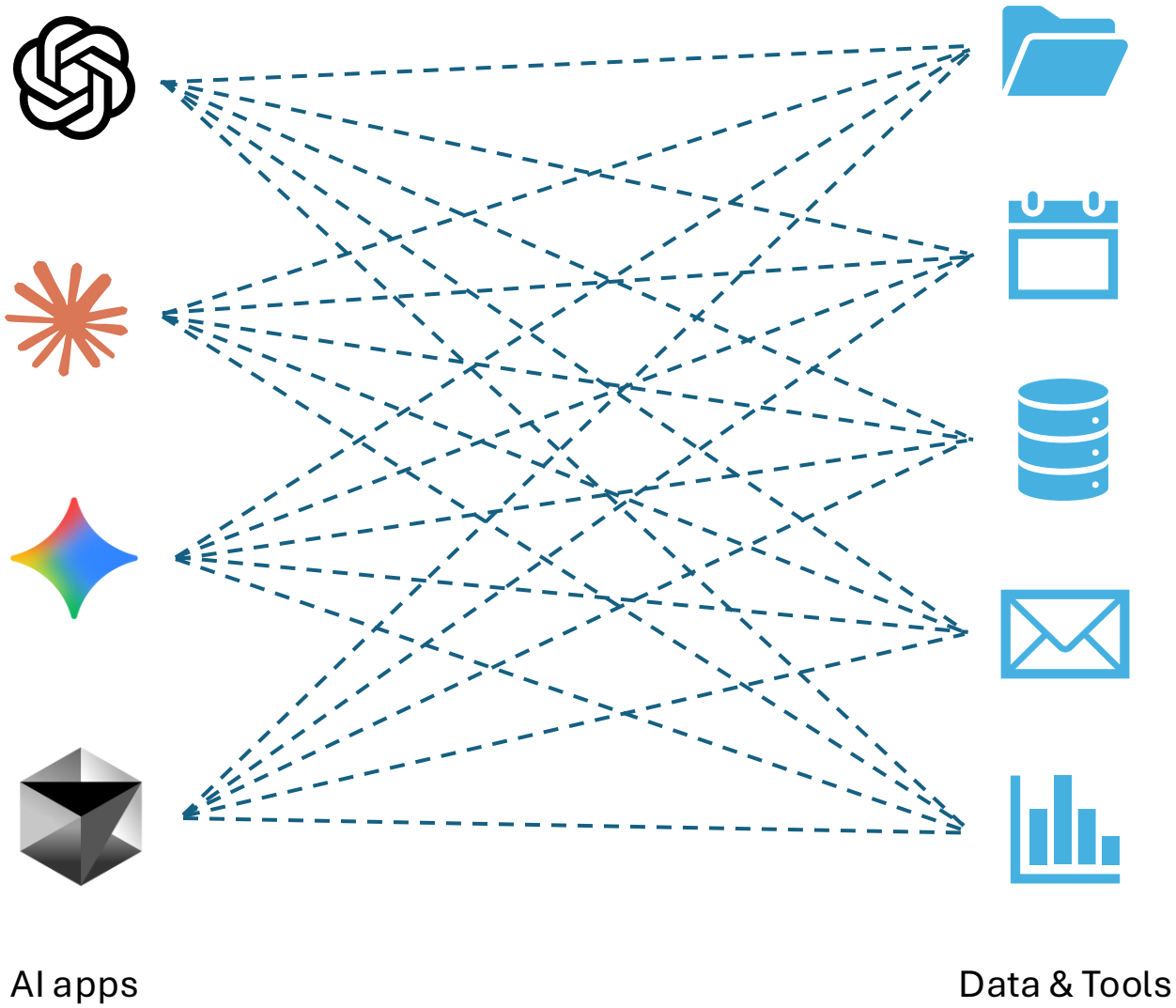
Build Phase

(~60 minutes)

What the heck is **Model Context Protocol**?

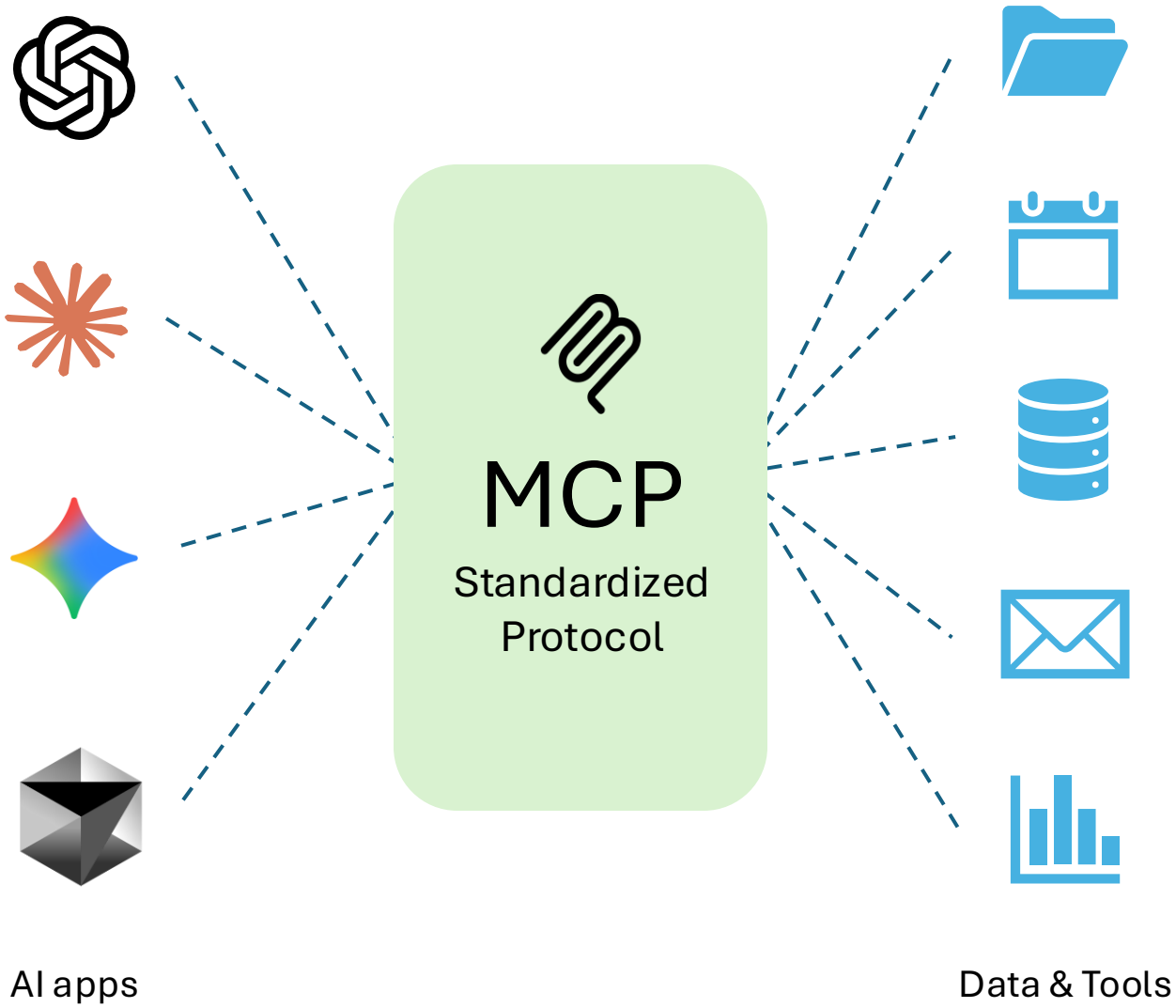
Traditional AI apps

- Training gaps, knowledge "cut-off" date
- Tool integration complexity
- Fragmented ecosystem



Model Context Protocol

- Open-source standard
- Seamless integration between AI apps/agents and external systems
  - Tools – search engines, calculators
  - Data sources – local files, DBs
  - Workflows – specialized prompts



Why should you care about MCP?

AI Engineer

- Connect AI to data, apps, and tools
- Less complexity, faster development

Tool Developer

- Build once, adopt everywhere
- One server, broad reach

End-User

- Richer, more powerful AI apps
- Better experience

Security Engineer

- Understand protocol, enforce controls
- Harden the attack surface

MCP architecture

MCP Host

IDE

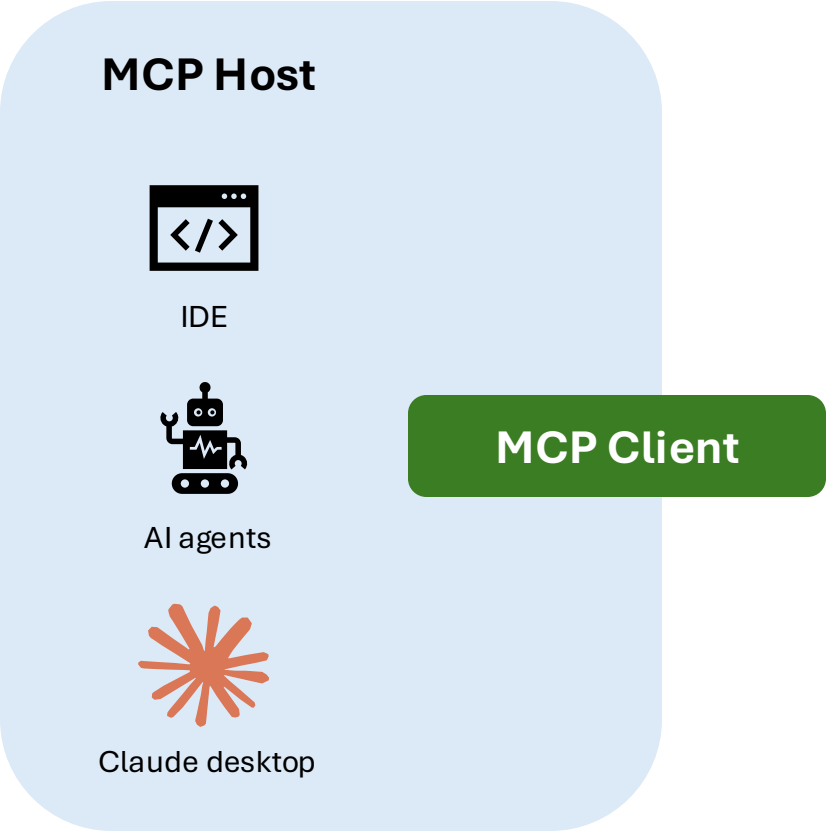
AI agents

Claude desktop

MCP Host

The AI application that coordinates and manages one or multiple MCP clients

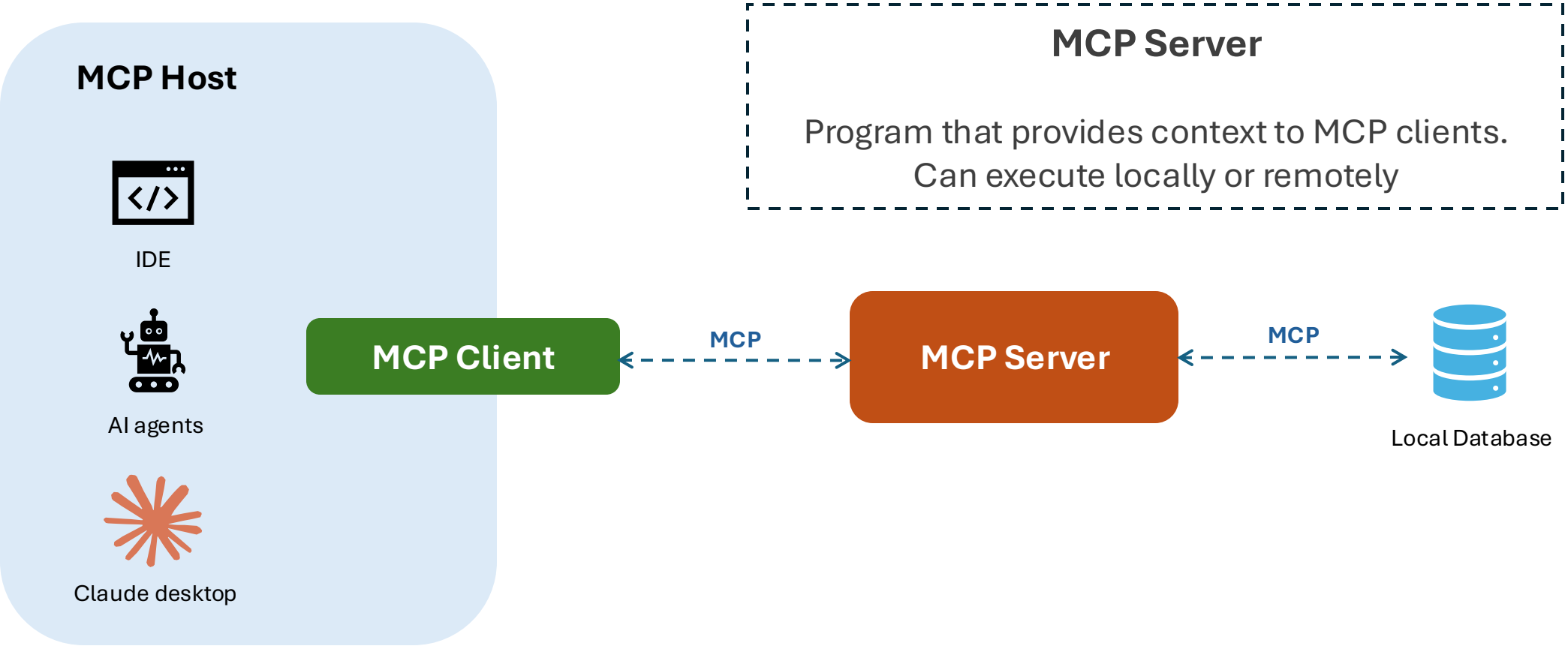
MCP architecture



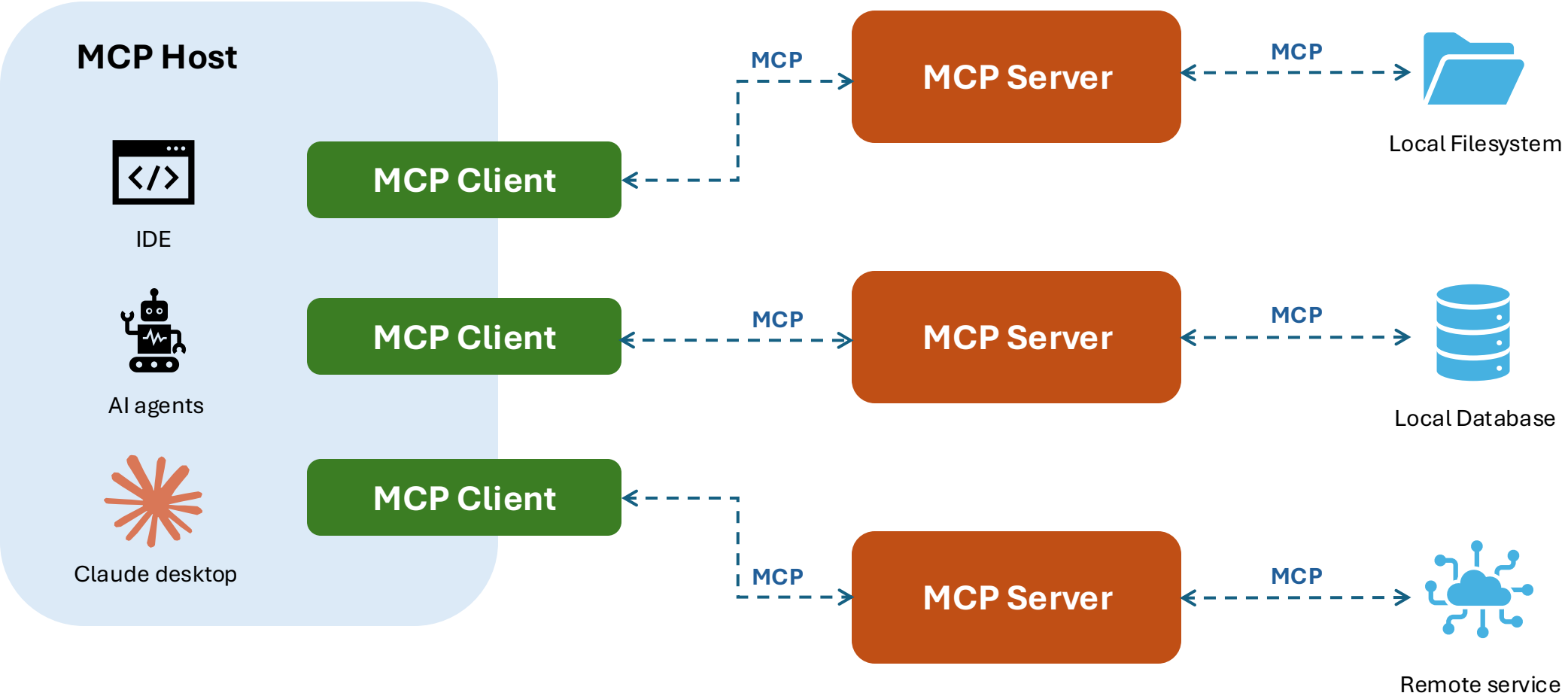
MCP Client

Component that establishes and maintains a connection to an MCP server, obtains context from the MCP server for the MCP host to use

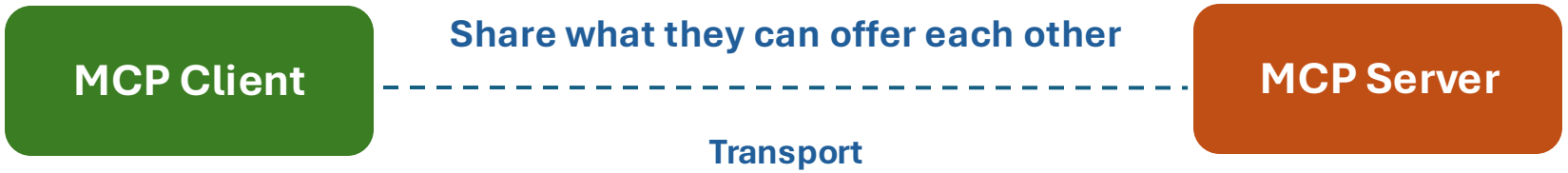
MCP architecture



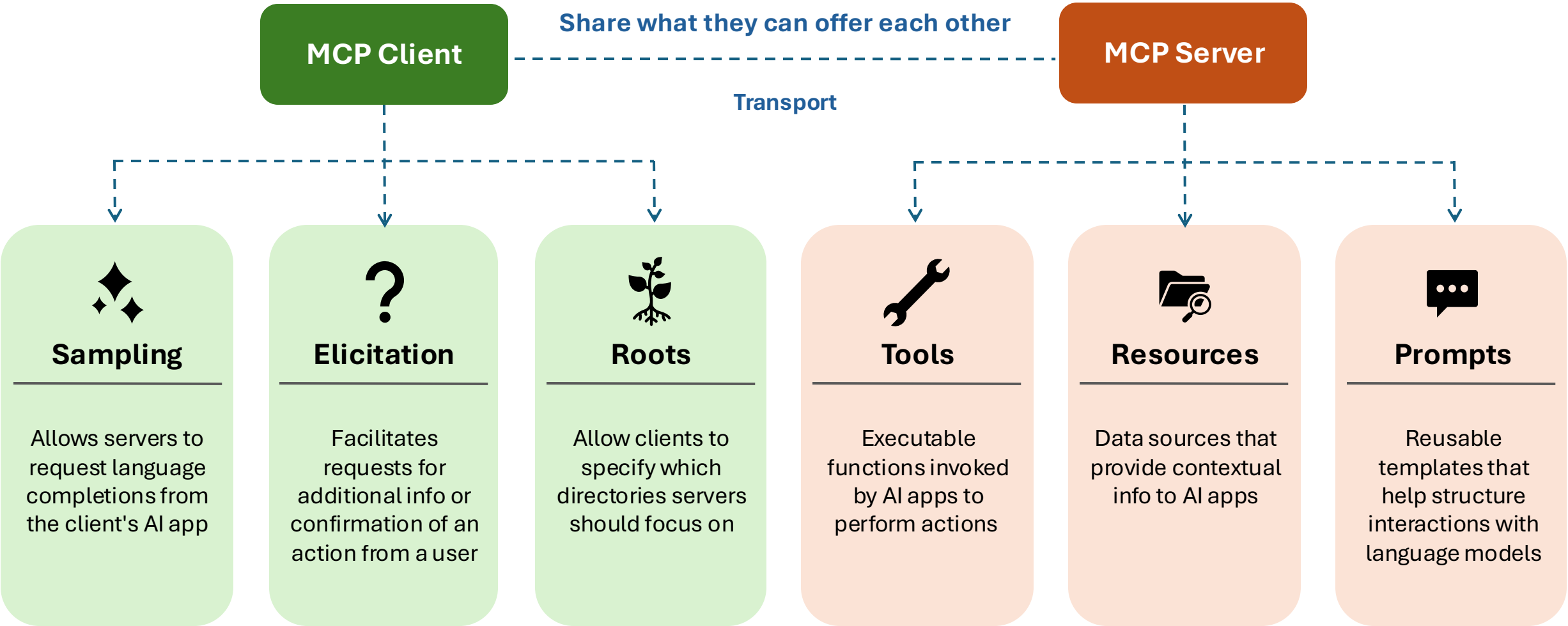
MCP architecture



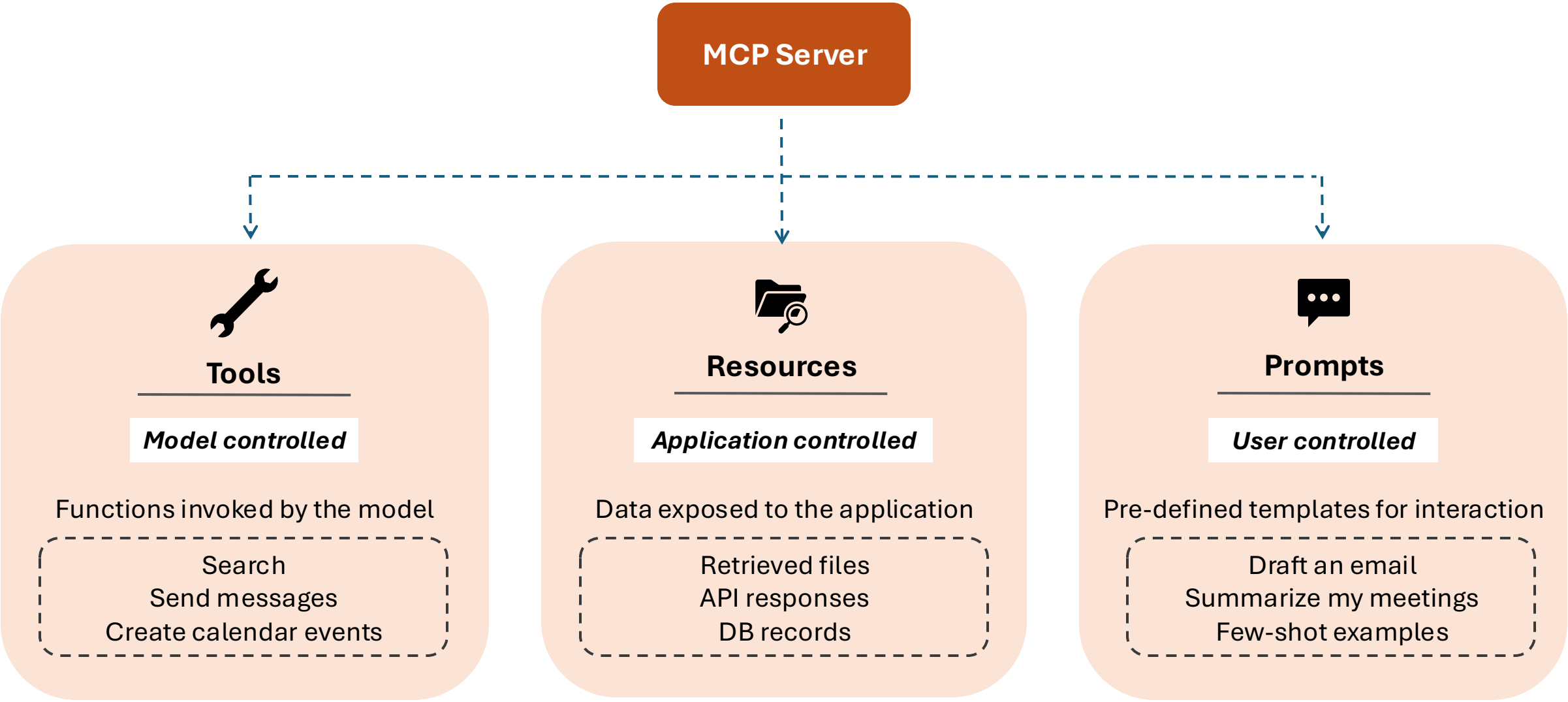
MCP primitives



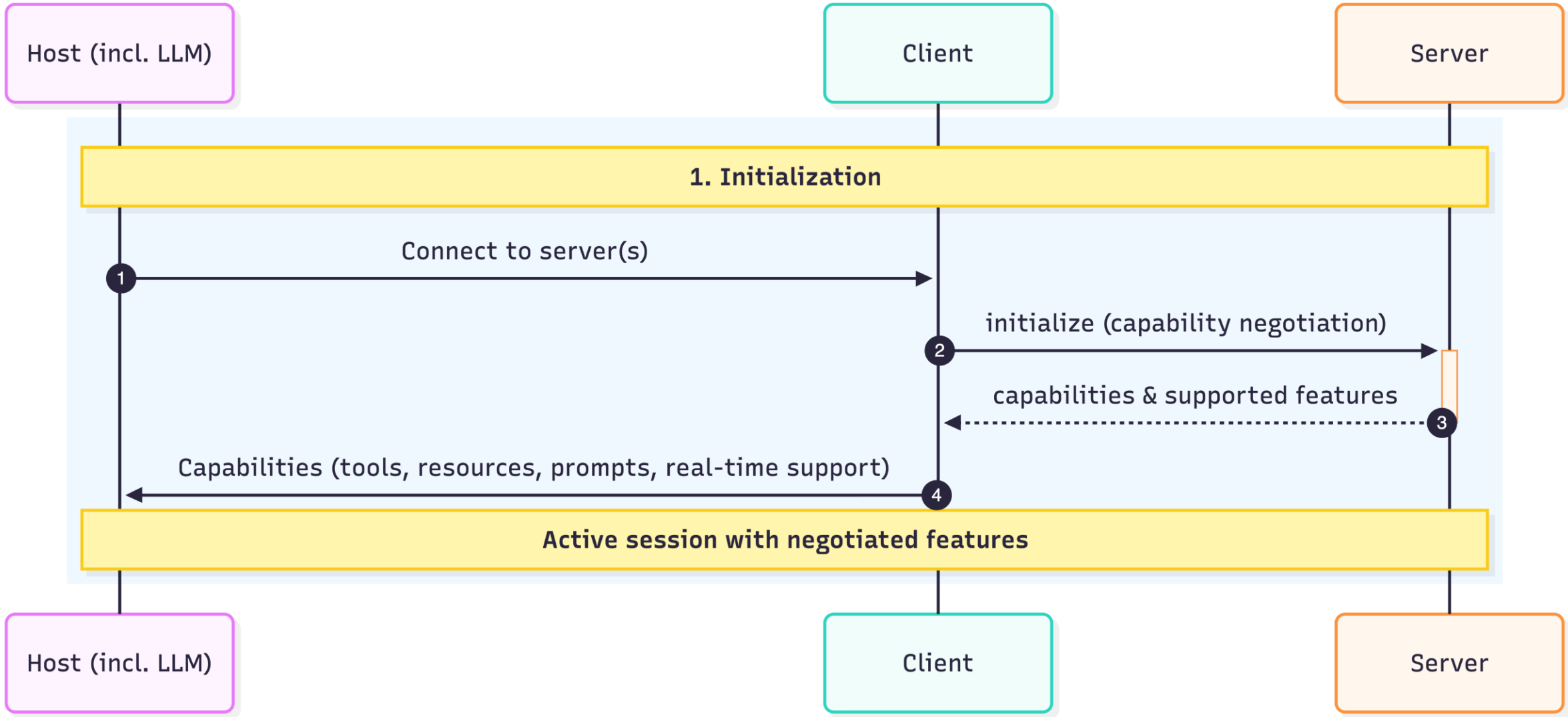
MCP primitives

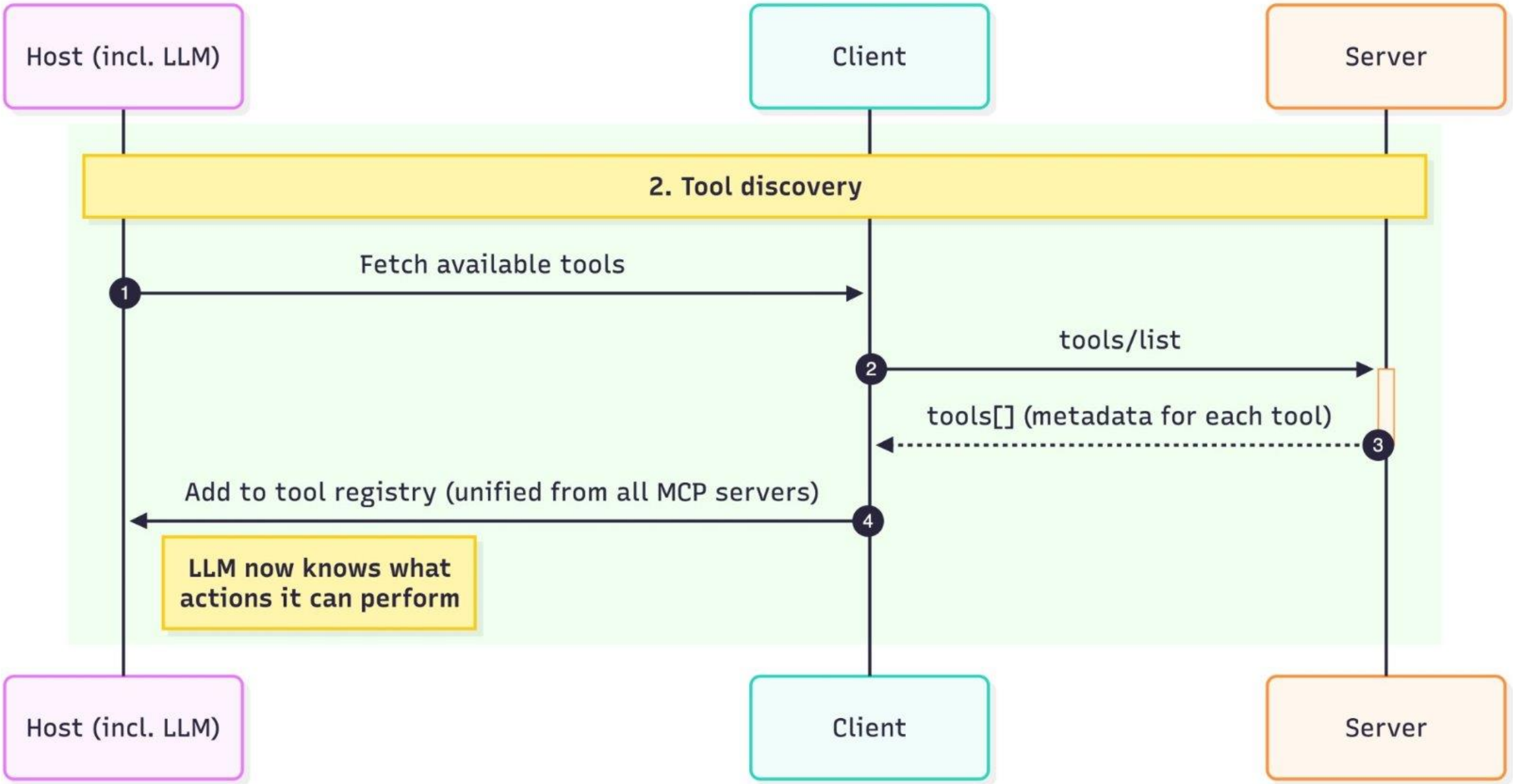


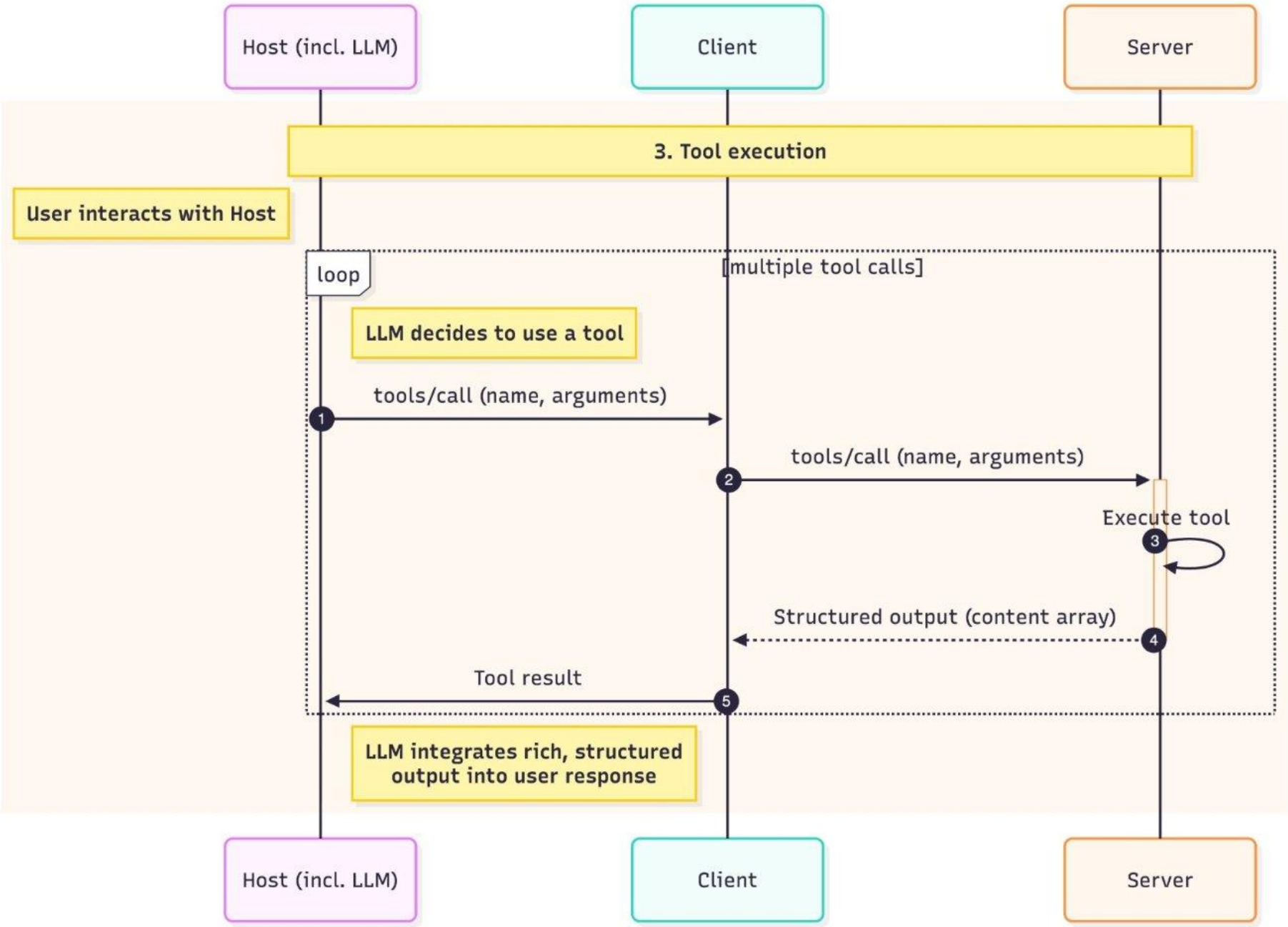
MCP primitives

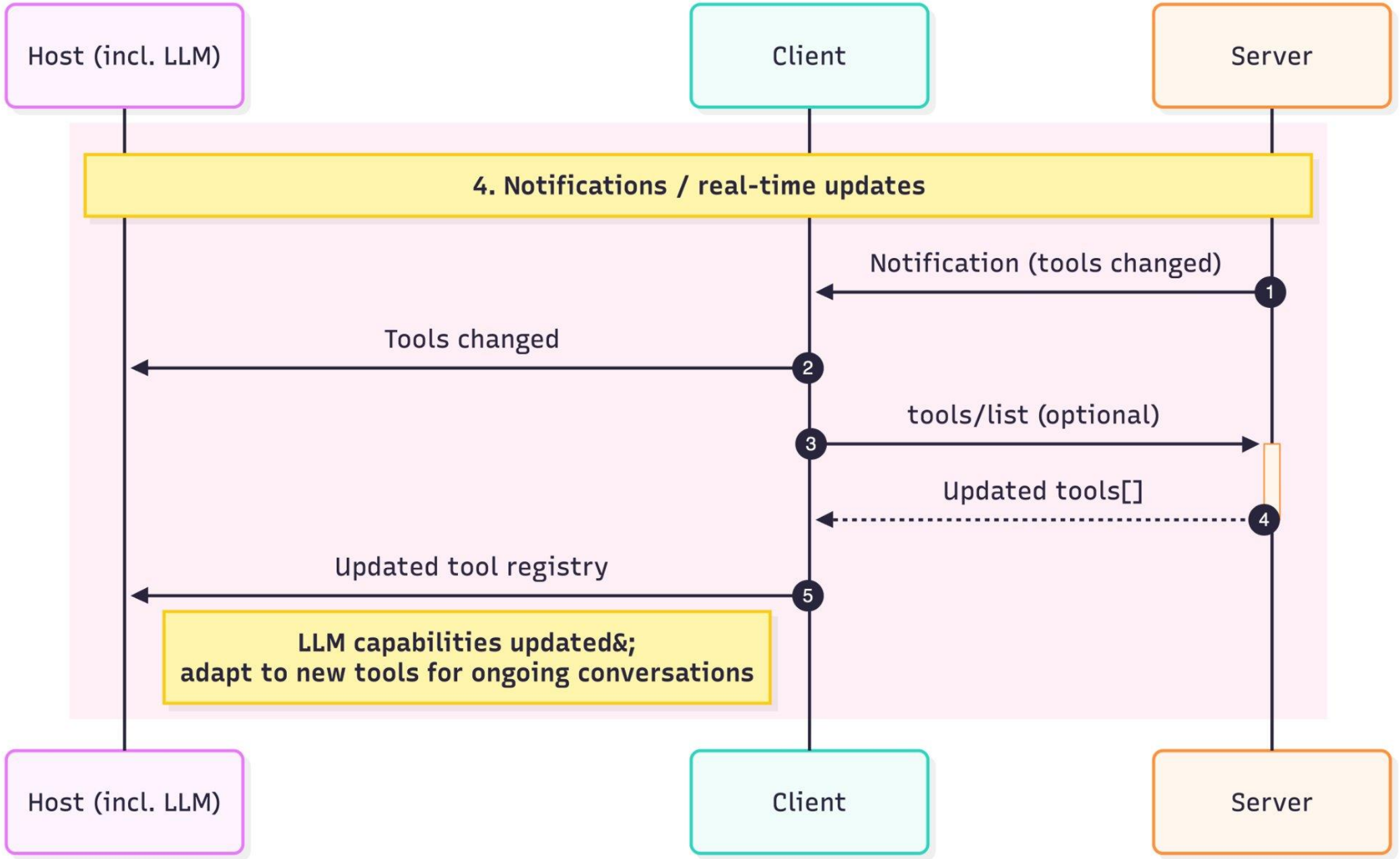


Walkthrough of a client server interaction

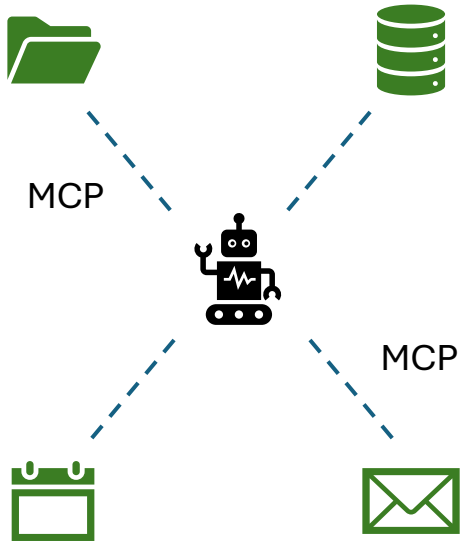

Host (incl. LLM)
Client
Server
1. Initialization
Connect to server(s)
1
initialize (capability negotiation)
2
capabilities & supported features
3
Capabilities (tools, resources, prompts, real-time support)
4
Active session with negotiated features
Host (incl. LLM)
Client
Server


Host (incl. LLM)
Client
Server
2. Tool discovery
Fetch available tools
1
tools/list
2
tools[] (metadata for each tool)
3
Add to tool registry (unified from all MCP servers)
4
LLM now knows what actions it can perform
Host (incl. LLM)
Client
Server


Host (incl. LLM)
Client
Server
3. Tool execution
User interacts with Host
loop
[multiple tool calls]
LLM decides to use a tool
tools/call (name, arguments)
1
tools/call (name, arguments)
2
Execute tool
3
Structured output (content array)
4
Tool result
5
LLM integrates rich, structured output into user response
Host (incl. LLM)
Client
Server


Host (incl. LLM)
Client
Server
4. Notifications / real-time updates
Notification (tools changed)
1
Tools changed
2
tools/list (optional)
3
Updated tools[]
4
Updated tool registry
5
LLM capabilities updated&;
adapt to new tools for ongoing conversations
Host (incl. LLM)
Client
Server

	MCP	VS	API
Purpose	AI-native, designed for LLMs		Software-to-software, general purpose
Invocation	Model decides what to use based on user query		Developer/system decides when/how to use it
Adaptability	Adapts to changes at runtime, dynamic nature		Breaks if code changes, needs development to update
Output types	Model interprets the output and integrates into response		Raw/structured outputs that require manual processing
Integration	Easier AI integration with adapters		Requires custom code for each API



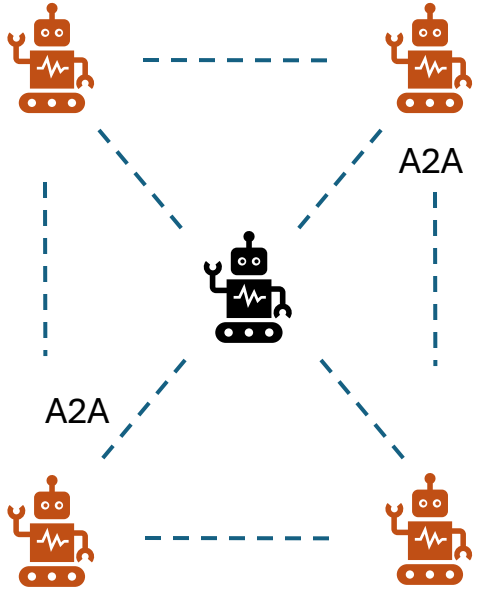
MCP

- How one agent functions
- Cognition
- Context

VS

A2A
(Agent to Agent)

- How multiple agents function
- Coordination
- Communication



Build time!

What we'll build today

- **Local Discovery** MCP server that helps you discover local food, events and activities of interest in a city

- The server exposes tools so you can search for -
  - Restaurants (Google Places) and various options
  - Events and venues (Ticketmaster discovery)

Setup and instructions

- Repo: https://github.com/hakunamatata-test/ship-fast-regret-faster
- Full Instructions: **/build/README.md**
- High-level steps:
  - Clone the repo above, follow install instructions
  - MCPJam setup
  - Google and Ticketmaster API keys
  - Run MCPJam locally
  - Add and run the Local Discover server
  - Build and have fun!!

New protocol, new problems

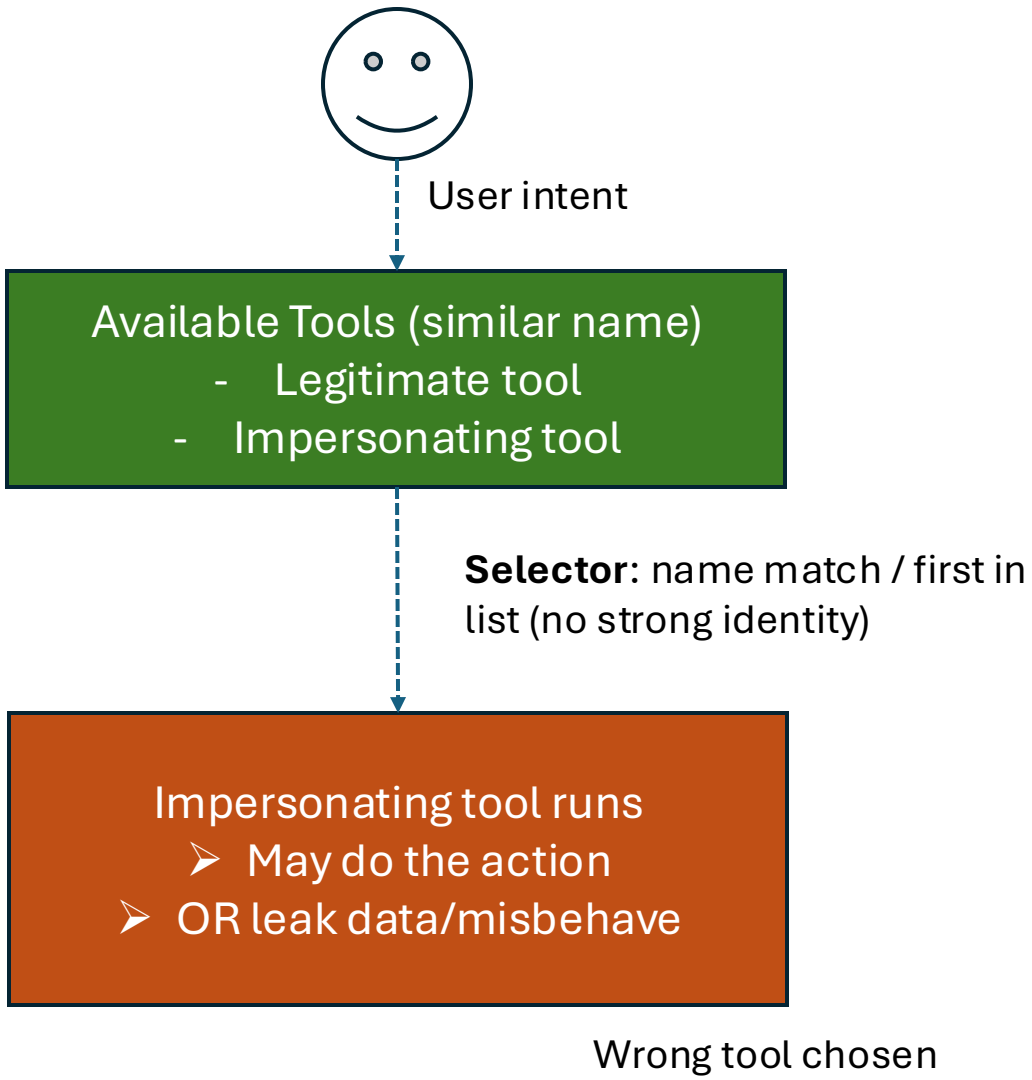
(~90 minutes)

When MCP becomes an attack surface

- Expands the attack surface beyond the model
- Model decides which tools to call
- Tools often have real-world permissions
- Shared context can be poisoned
- Trust assumptions are easy to break

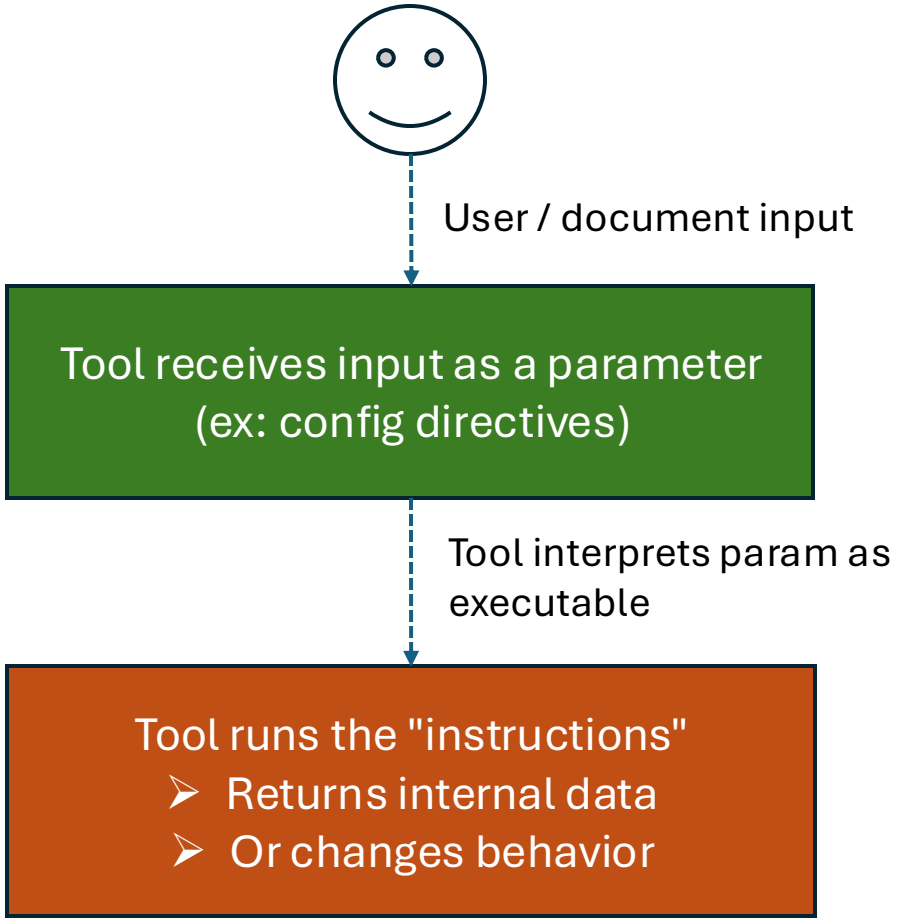
Naming/Impersonation

An attacker registers a tool with a name similar to a legitimate one. The system picks which tool to run by name or list order, not by a strong identity, so the wrong (attacker's) tool can be chosen. That tool may do the action but also leak data or misbehave.



Prompt Injection

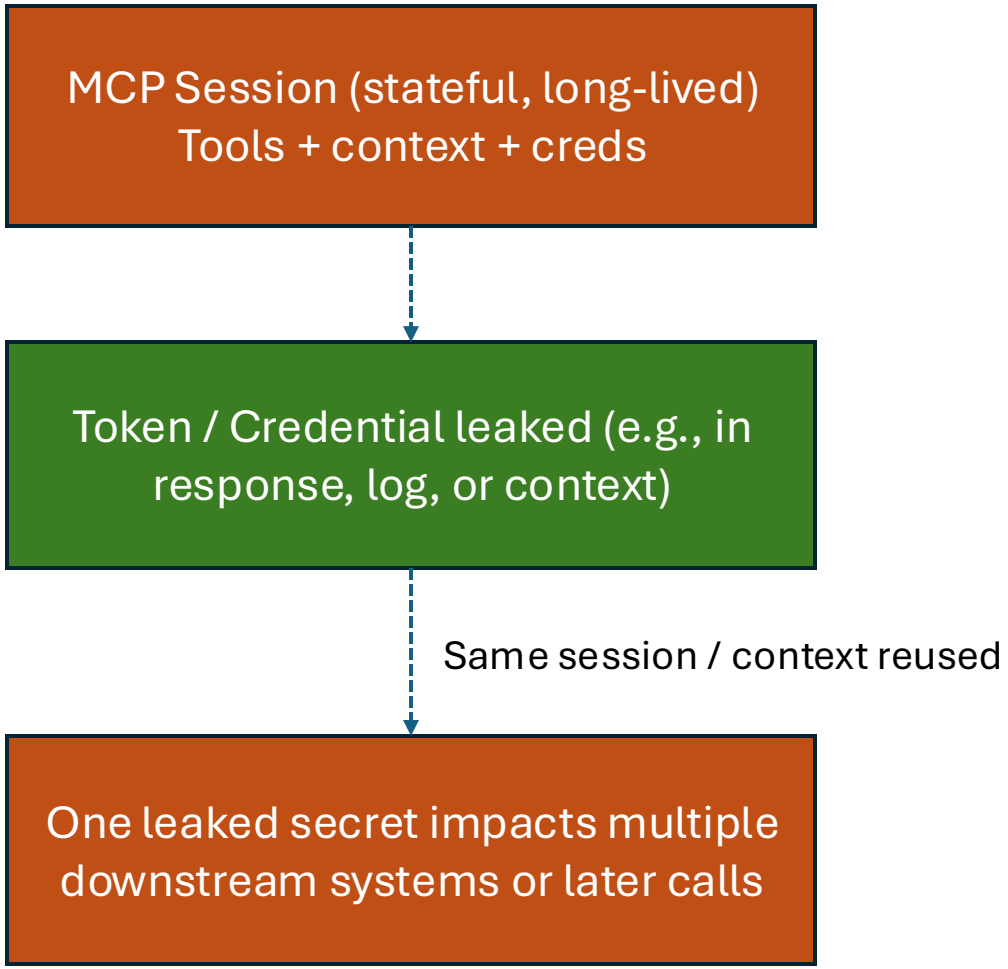
Untrusted input (e.g. from the user or a document) is passed into a tool as a parameter. The tool treats that parameter as instructions or config and runs it. An attacker embeds “instructions” in the data; the tool executes them and can return internal data or change behavior.



Secret Exposure

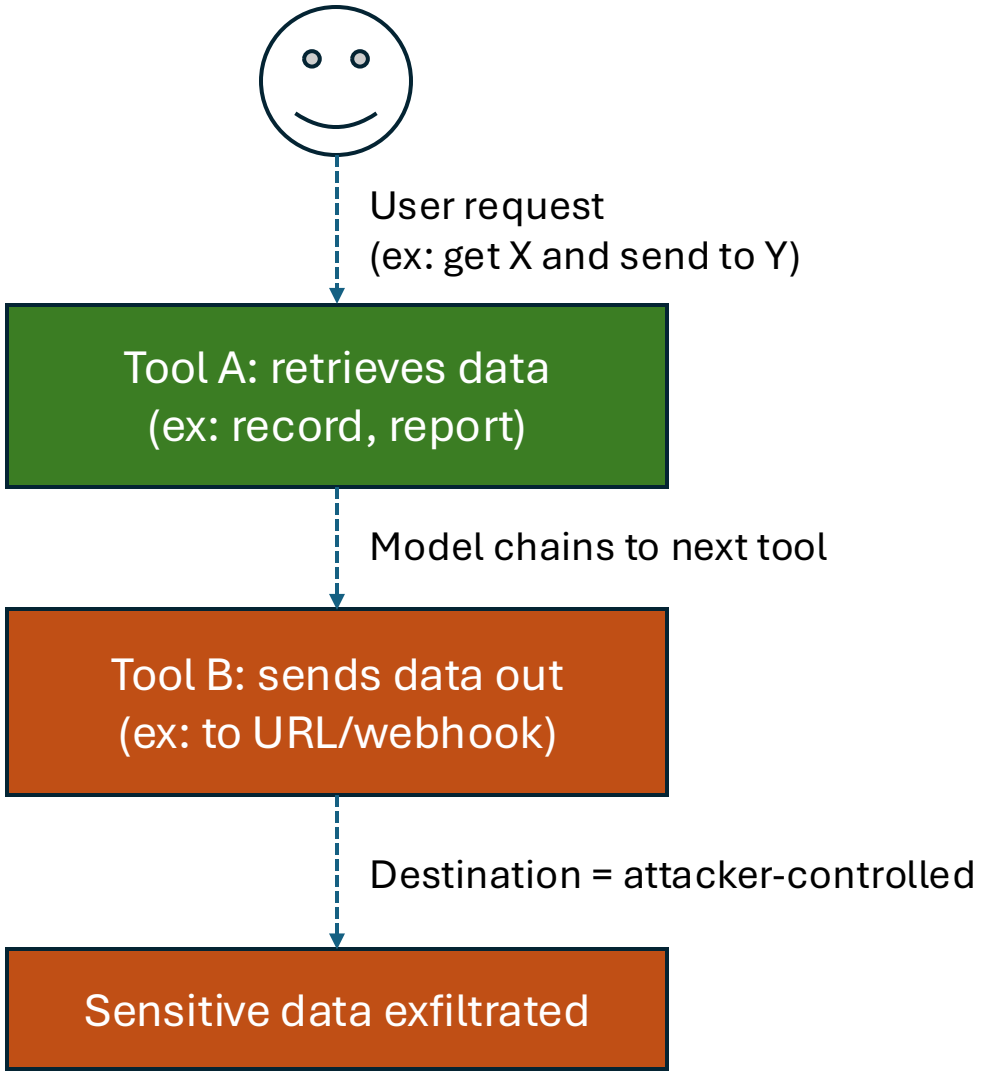
MCP systems often mishandle sensitive tokens and credentials, e.g. leaking them in tool responses, logs, or context.

The protocol enables long-lived sessions, stateful agents, and contextual persistence, so one leaked secret can impact multiple downstream systems and persist across interactions.



Tool chaining

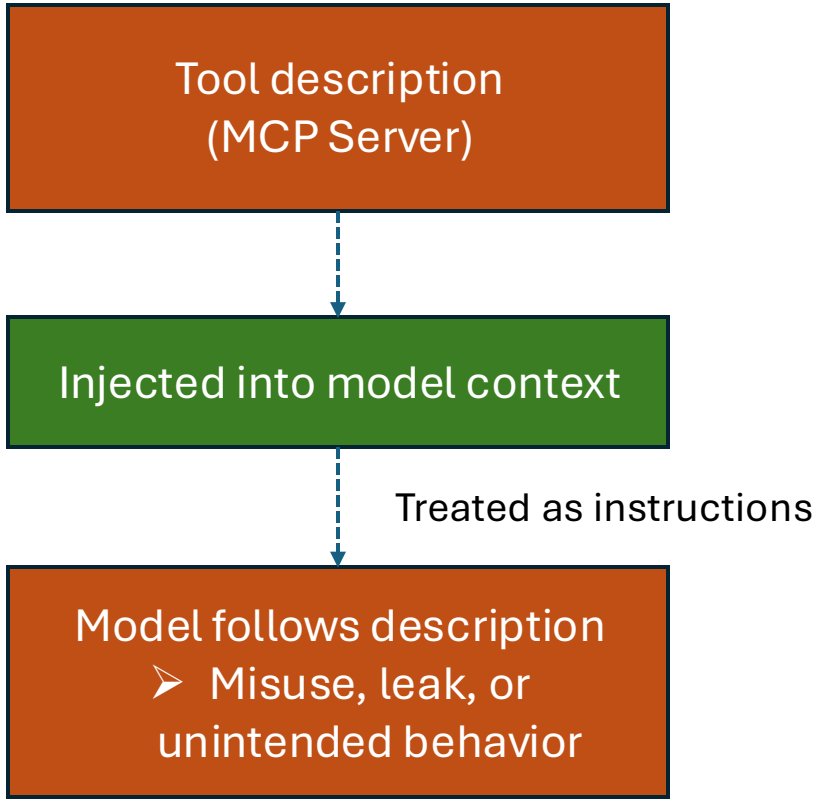
Individually safe tools, e.g. one that retrieves data, one that sends data to a URL, can be chained by the model on a single user request. Without policy on what may be chained and where data may be sent, the user can trigger a chain that exfiltrates sensitive data to an attacker-controlled endpoint.



Tool Poisoning + MCP Rug Pull

Tool descriptions in MCP become executable instructions inside the model's context, creating a powerful injection surface.

In a rug pull, a server silently modifies a previously approved tool to introduce malicious behavior after trust has already been established.

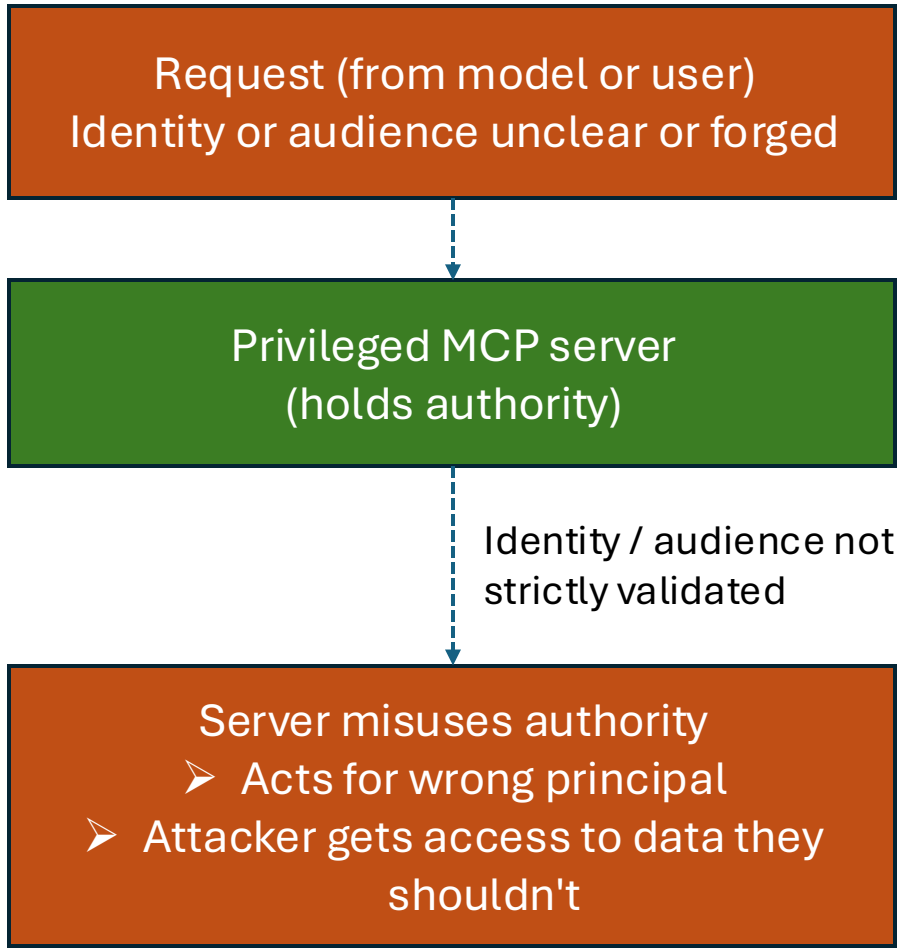


Confused Deputy

A privileged MCP server holds authority (e.g. to access data or call downstream services).

It can be tricked into misusing that authority, e.g. by accepting a request that appears to come from a legitimate user or context but was influenced by an attacker.

If user identity and token audience are not strictly validated, the server may act on behalf of the wrong principal or for a scope the caller is not allowed to use, so attackers can access data they shouldn’t.



Hacking Time!

Setup and instructions

- Repo: https://github.com/hakunamatata-test/ship-fast-regret-faster
- Full Instructions: **/break/README.md**
- High-level steps:
  - Clone the above repo
  - Install uv
  - MCPJam account setup (for model access)
  - Install Node.js and npm for MCPJam
  - Run MCPJam locally
  - Add and run each challenge server
  - Have fun!

Live Walkthroughs

Defend Phase

(~60 minutes)

Primer on Threat Modeling

- Threat Modeling is a way for us to understand systems deeply enough so that we can predict where they could go wrong

- **Adam Shostack's four-question Model:**
    - What are we building?
    - What can go wrong?
    - What are we going to do about it?
    - Did we do a good job?

Threat Modeling Frameworks

STRIDE

- **S**poofing
- **T**ampering
- **R**epudiation
- **I**nfo Disclosure
- **D**enial of Service
- **E**levation of Privilege

MAESTRO

- **M**emory (poisoning)
- **A**utonomy (unexpected behavior)
- **E**mbodiment (tool misuse)
- **S**ocial Engineering (promptinjection)
- **T**rust Shift (delegated identity)
- **R**untime Dynamics (chainingrisks)
- **O**rchestration (multi-agentworkflows)

Applying STRIDE to MCP

- **S**poofing – Fake/rogue MCP servers, Tool name impersonation
- **T**ampering – Tool response manipulation, Context poisoning
- **R**epudiation – Unlogged tool invocations, Missing audit trails
- **I**nfo Disclosure – Excessive tool permissions, Cross-server data leakage
- **D**oS – Tool flooding, Recursive tool chaining that never terminates
- **E**levation of Privilege – Permission boundary violation, Tool chaining escalation

Threat Modeling Process

Map

- **Map Architecture**
- Draw your MCP components – servers, agents, data flows, trust boundaries

Identify

- **Identify Assets**
- What's valuable? Customer data, API keys, Admin functions?

Apply

- **Apply STRIDE or a framework of your choice**
- Walk through each threat category systematically for each component

Prioritize

- **Prioritize Risks**
- Focus on high-impact, high-likelihood

Design

- **Design Mitigations**
- Layer defenses to address top risks

Defenses: Naming

Use stable tool identity

- Resolve tools by a stable ID or registry entry, not by name or substring match.

Allowlist trusted tools

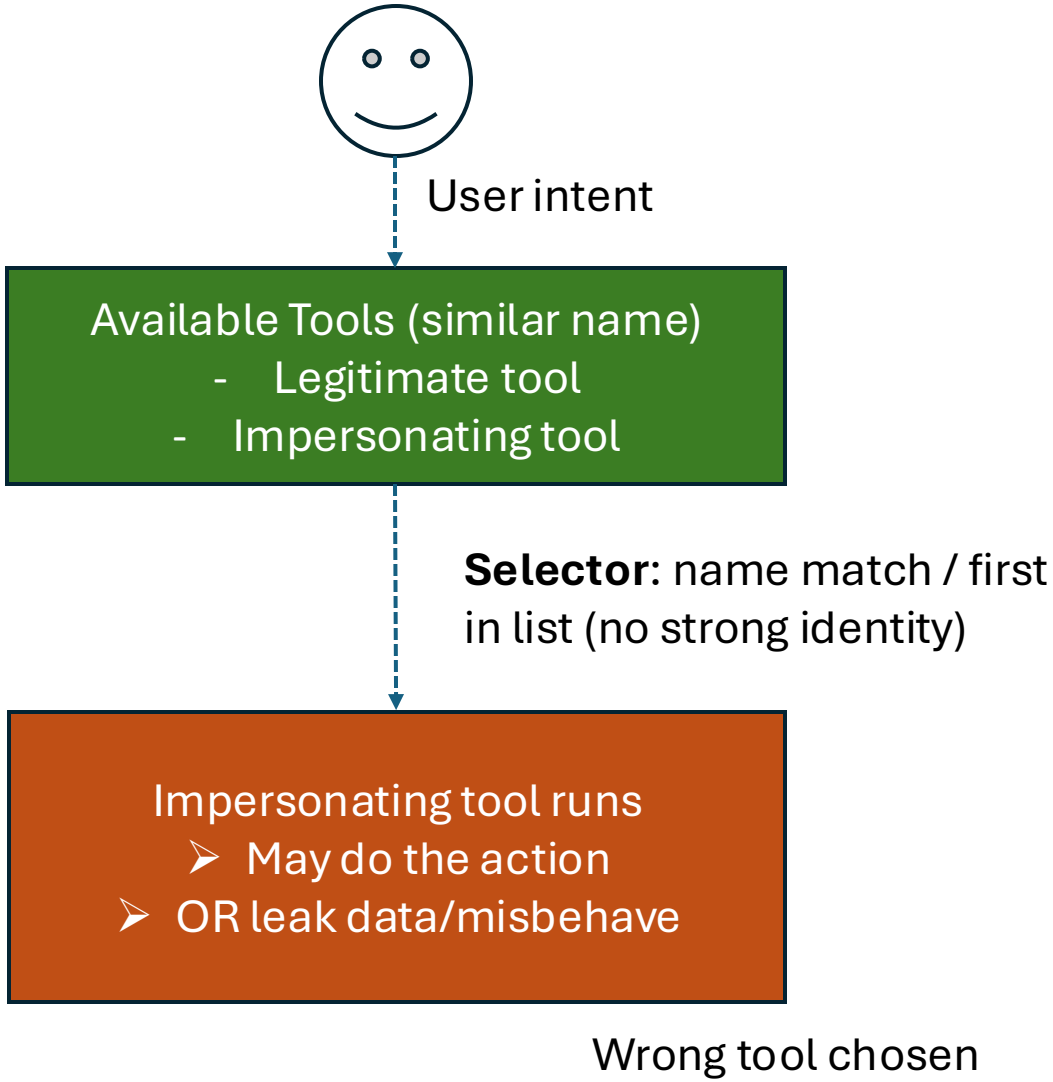
- Maintain and explicit allowlist of tool IDs that are approved for a given use.

Limit who can register servers

- Restrict which MCP servers can be added.

Audit tool list

- Log and periodically review which tools are available to the model.



Defenses: Prompt Injection

Strict separation of data and instructions

- Never pass user or document content directly into a tool parameter that controls behavior.

Validate and sanitize parameters

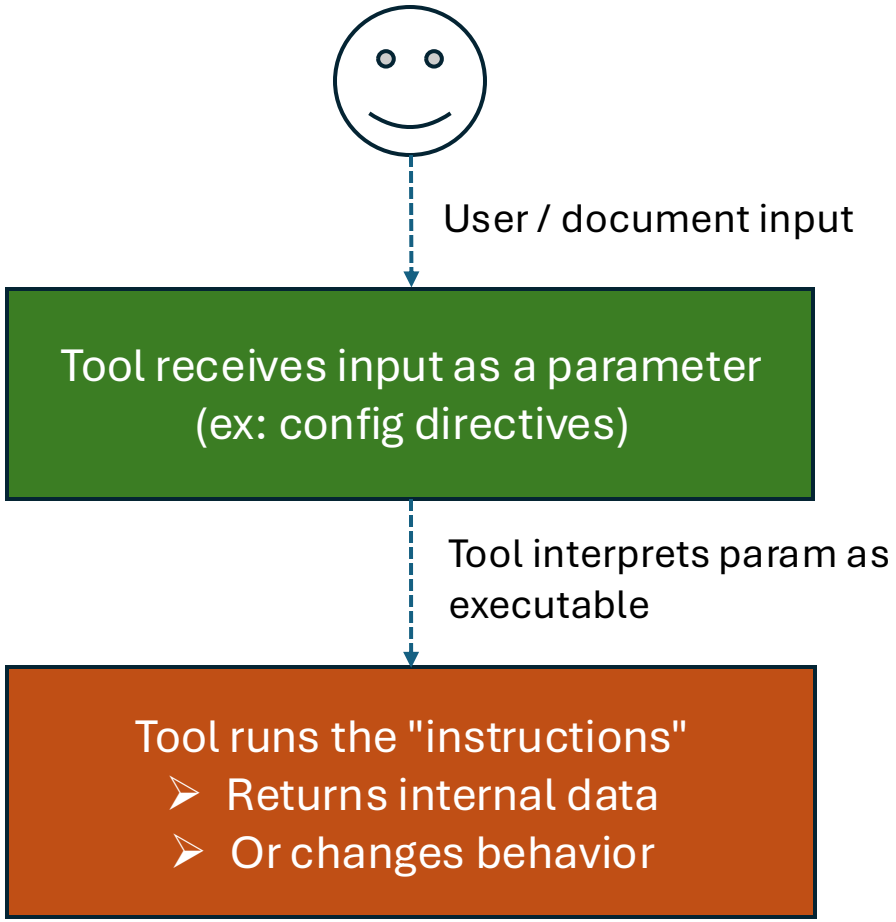
- If a parameter can contain user content, validate format and length; reject or escape content that looks like instructions.

Least privilege for tool behavior

- Tools should expose only the minimum options needed.

Log and monitor

- Log tool params (redact secrets); monitor for anomalous or injection-like patterns.



Defenses: Secret Exposure

Don't leak tokens or internal refs

- Ensure tool responses, logs, and context never include credentials, internal IDs, or debug fields.

Scoped credentials and short-lived tokens

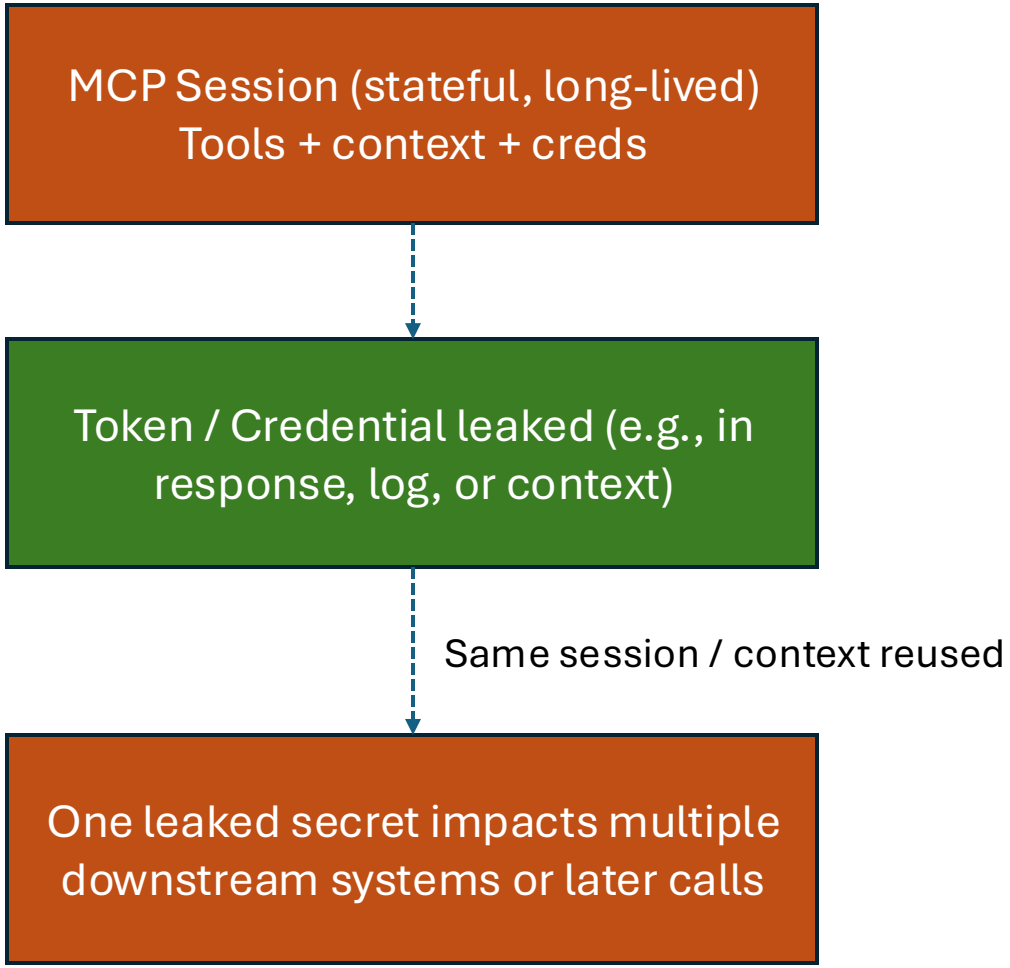
- Use tokens with minimal scope and short TTL; rotate regularly so one leak has limited impact.

Access control on sensitive operations

- Any tool that returns sensitive data must validate the caller's identity and authZ.

Segment context

- Limit what stays in long-lived session or context; avoid persisting secrets across many turns or downstream systems.



Defenses: Tool chaining

Policy on tool composition

- Define which tools may be chained (ex: "get record" may not be followed by "send to URL" without a separate approval or gate).

Allowlist outbound destinations

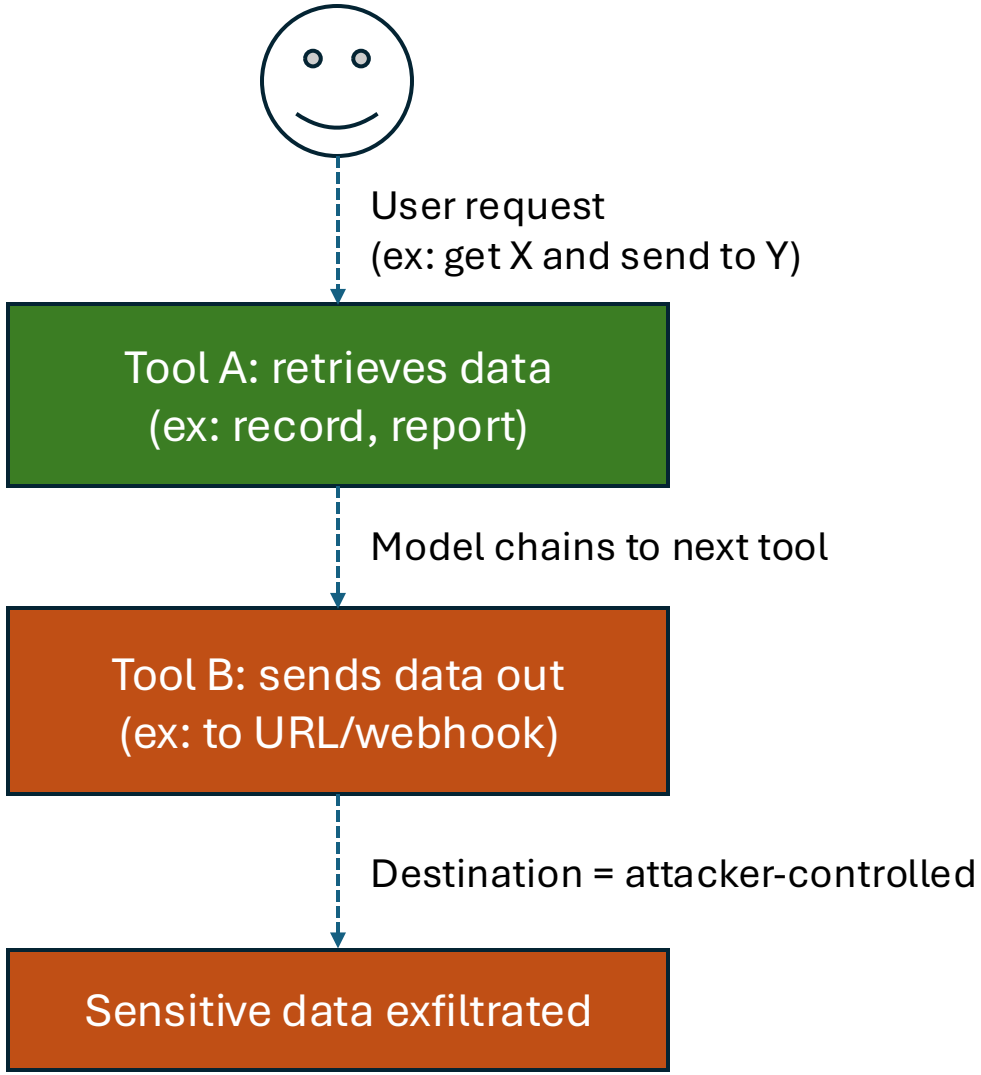
- Only allow send/webhook-style tools to call allowlisted URLs or domains; block attacker-controlled endpoints.

Restrict payload content

- Limit what data can be included in outbound calls (ex: no PII or full records unless explicitly allowed).

Audit chained calls

- Log when multiple tools are used in one flow; alert on high-risk combos (ex: data fetch + external send).



Defenses: Tool Poisoning + MCP Rug Pull

Treat descriptions as trusted input

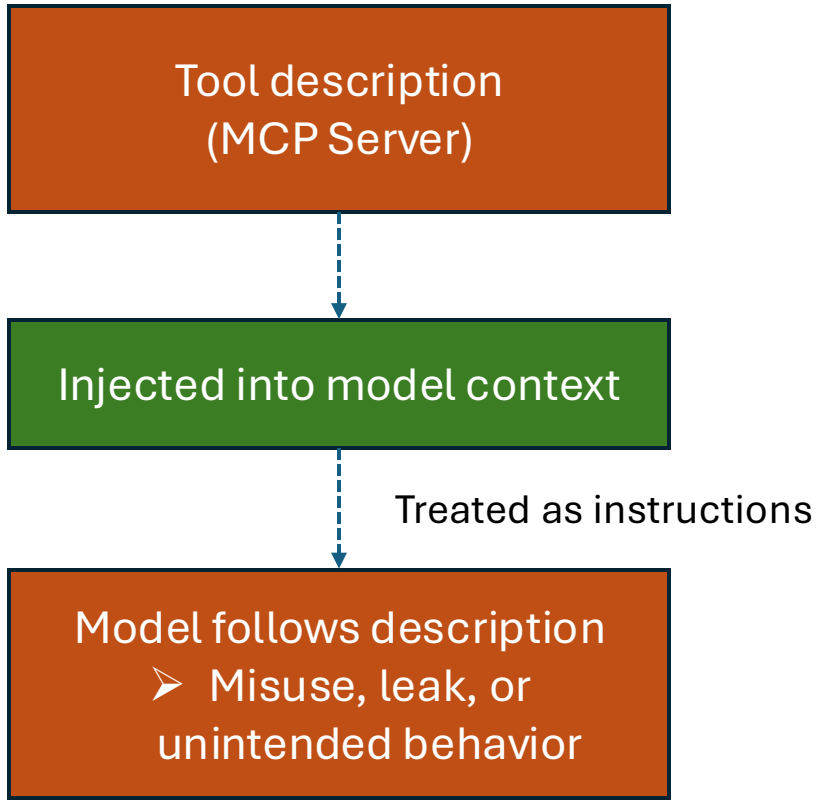
- Tool descriptions are executable in the model's context. Source them from trusted, reviewed content; don't allow arbitrary server-supplied text to become the only description.

Integrity and provenance

- Where possible, pin or sign tool definitions (name + description + schema); detect when a previously approved tool's definition changes (rug pull)

Review and allowlist servers

- Only connect to MCP servers that have been reviewed; treat new or updated tools as a change that may need re-approval.



Defenses: Confused Deputy

Strict identity and audience validation

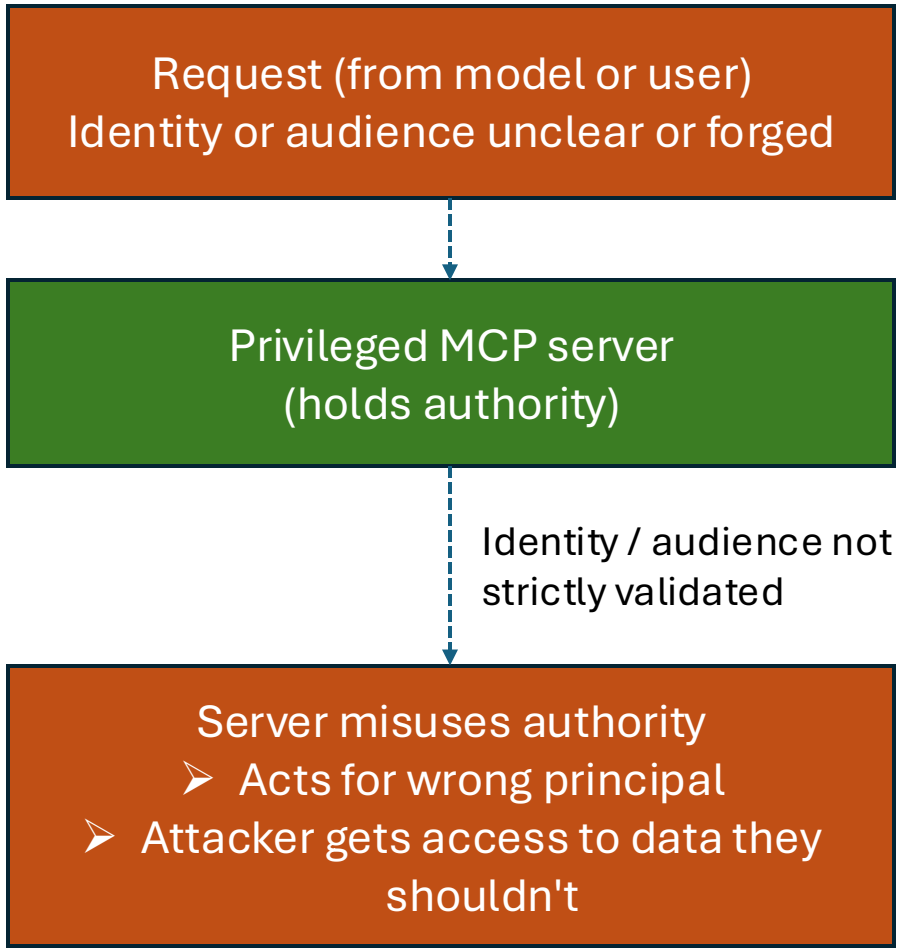
- Before acting, validate the identity of the caller and the audience of any token. Reject requests where identity is missing, wrong, or not allowed for the requested scope.

Principle of least privilege

- MCP servers should hold only the authority they need; use scoped tokens and avoid overly broad delegation.

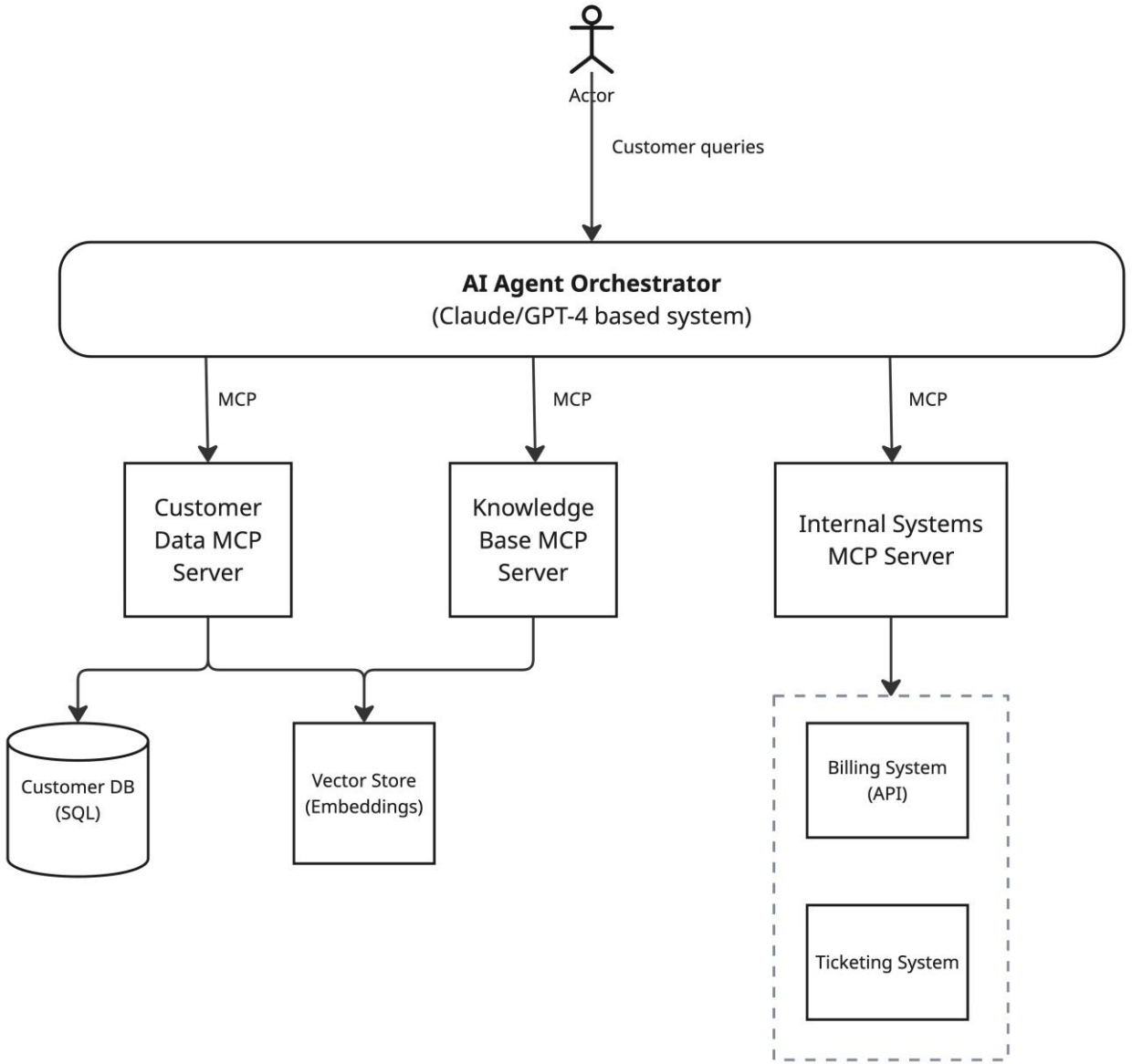
Explicit authZ checks

- For every sensitive operation, check "is this caller allowed to perform this action for this principal/resource?"



Exercise Time!

Sample MCP Architecture for Threat Modeling



Exercise Instructions

Objective

Conduct a threat modeling exercise on the provided sample architecture. Identify potential threat scenarios and recommend security controls to mitigate associated risks. Each group will present a brief walkthrough of their findings.

- Time: ~30 minutes
- Output:
  - **Threats**: What can go wrong?
  - **Mitigations**: For each threat (or group of threats), what would you put in place to reduce or prevent it
- Have one person per table nominated for the final readout.

Wrapping up

Workshop Recap

Build

- MCP concepts, architecture, primitives
- Develop and test tools in an MCP server

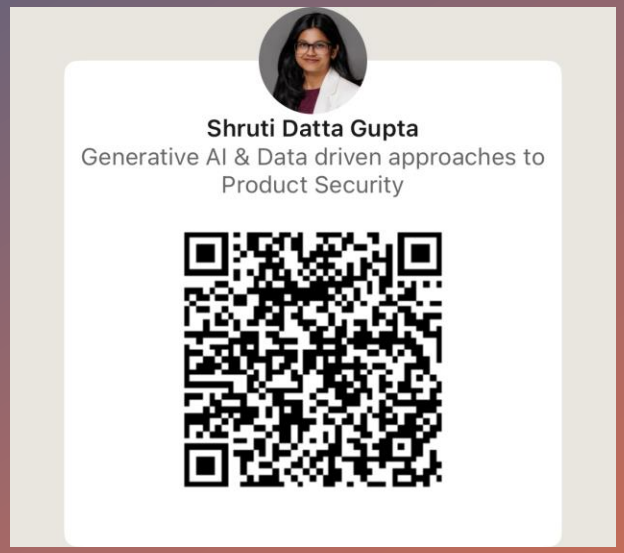
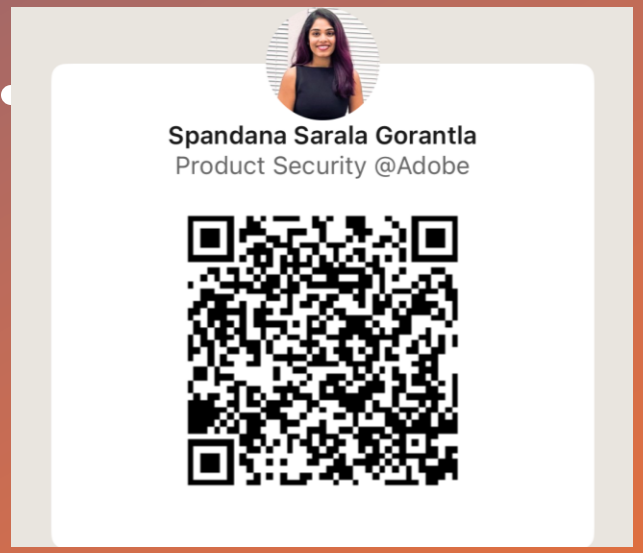
Break

- Common MCP attacks & mitigations
- CTF style break exercises

Defend

- Applying threat modeling to MCP workflows
- Live MCP threat modeling exercise

Learnings/Takeaways

- MCP expands capability, and the attack surface
- Model-driven tool use changes traditional trust boundaries
- Seemingly "small" design decisions create big security risks
- Secure MCP requires intentional design, not patching later

Thank You! Connect with us :)